

Concept for a Multi-Scale Distributed Acoustic Sensing Network

Bjørn Remseth
Email: la3lma@gmail.com
With AI assistance

Abstract—This concept document describes a distributed acoustic sensing network that can operate across several geographic scales: local wired microphone groups spanning meters, node clusters spanning football-field to campus scale areas, and geographically separated clusters spanning kilometers to hundreds of kilometers. The motivating target is not merely to collect audio, but to preserve enough timing and spatial information that independent observations can be combined into a shared acoustic map. The document defines the problem, identifies engineering constraints, and outlines a candidate systems architecture from clock distribution through local sampling, acoustic feature extraction, network payloads, GPU-assisted spatial inference, hardware architecture, and field test strategy. It is intended to support later formulation of a focused product requirements document, not to serve as a final product requirements document itself.

Index Terms—distributed acoustic sensing, microphone arrays, clock synchronization, drone detection, beamforming, voxel mapping, edge processing

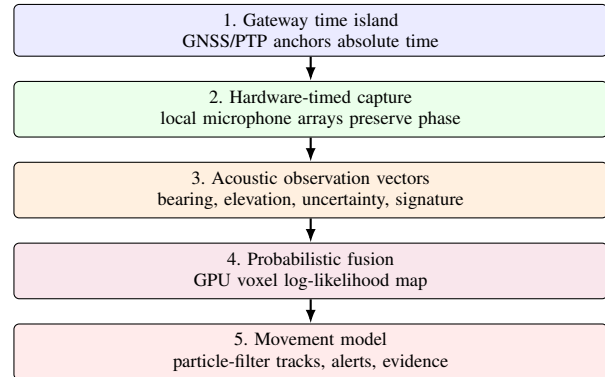
version: 2026-06-21-14:03:45-CEST-839de3c-main

Note on document creation: This document was created with AI assistance. See the “About This Document” section for details on the methodology and verification processes used.

Context and status: This work began as a Sunday-morning quick design exercise after reading NATO’s public page on the Defence Innovation Accelerator for the North Atlantic (DIANA) online. It should be read as an exploratory systems-design sketch: interesting material for further development, not a fully mapped concept paper, procurement recommendation, or done-done design.

I. EXECUTIVE TL;DR

Conclusion: Build the system as a time-disciplined acoustic observation network, not as a raw-audio recorder network. Each cluster should establish absolute time locally, local nodes should capture phase-coherent audio in hardware, nodes should send compact observation vectors rather than continuous multichannel audio, and gateways or regional processors should fuse those observations into spatial likelihood maps and target tracks.



At-a-glance system concept: preserve timing at capture, compress local audio into spatial observations, then fuse observations into maps and target tracks.

Decision	What it means
Time is hardware	Do not rely on WAN, router NTP, packet arrival, or user-space timestamps for acoustic phase.
Audio is local	Capture synchronized 48 kHz windows at the node; run local TDOA/DOA and keep raw audio selective.
Network payloads are semantic	Send bearing, elevation, variance, clock quality, environment, and compressed evidence.
Fusion is probabilistic	Add observation cones into a voxel log-likelihood map; report uncertainty, not just points.
Movement is tracked	Use particle filters or similar Bayesian trackers for moving sources with nonlinear dynamics.

The proposed path is therefore: prototype a small time-disciplined microphone node, prove observation-vector generation in controlled tests, build a gateway that maintains the timing root and observation archive, then add GPU-assisted map fusion and target tracking. The rest of this document explains the reasoning, constraints, candidate technologies, and validation path behind that conclusion.

II. PURPOSE AND SCOPE

This document captures a system concept for a distributed acoustic sensor network whose sensor clocks should be synchronized to approximately $1/64000$ seconds, or $15.625 \mu\text{s}$. The concept extends that timing target into a broader architecture for synchronized audio sampling, acoustic source localization, and drone-oriented detection.

The document should be read as a design exploration. It is intentionally more concrete than a brainstorm, because it

follows through to candidate hardware and software structure. It is intentionally less rigid than a product requirements document, because enrichment, quantitative verification, trade studies, and critique are expected to happen later. The immediate goal is to preserve the line of reasoning in a form that can become a well-informed PRD.

The primary design question is:

How can a distributed network of battery-powered and gateway-powered acoustic sensors preserve enough shared time, phase, and geometry to detect, localize, classify, and map sound sources across areas ranging from a football field to many square kilometers, and potentially to multiple geographically separated regions?

III. SYSTEMS DESIGN VIEWPOINT

The system should be treated as a system of interacting elements rather than as a hardware project with software added later, or a software project waiting for sensors. This follows the systems-engineering viewpoint used in ISO/IEC/IEEE 15288 and the INCOSE and NASA systems engineering handbooks: the object of design is the whole life-cycle system, including interfaces, verification, deployment, operations, maintenance, and retirement [1], [2], [3].

For this concept, the key system elements are:

- acoustic transduction and local array geometry,
- time sources, oscillators, clocks, triggers, and timestamp paths,
- node firmware and local DSP,
- node power, enclosure, thermal, and weather protection,
- cluster networking and timing topology,
- gateway time anchoring, storage, and backhaul,
- observation schemas and data provenance,
- regional GPU fusion, classification, and tracking,
- operator displays, evidence workflows, and alert policy,
- field calibration, validation, and sustainment.

The important engineering work is mostly at the interfaces. A microphone capsule is only useful if its physical position, clock path, noise floor, and calibration are known. A gateway time source is only useful if the uncertainty is carried into the observation vector. A GPU voxel map is only useful if node geometry, environmental state, observation covariance, and classifier confidence are represented coherently. This document therefore frames hardware and software decisions as coupled system trades.

IV. PROBLEM DEFINITION

The system is meant to observe sound sources over multiple spatial dimensions rather than in a single local microphone array. The relevant scales are:

- 1) **Local sensor scale:** meters. One node may have one microphone, but the more useful form is a small wired group of microphones. The local array should sample phase-coherent audio and estimate local direction-of-arrival or time-difference-of-arrival features.

- 2) **Node cluster scale:** tens to hundreds of meters, perhaps up to the size of a football field or several adjacent fields. Nodes may be battery-powered, may use Wi-Fi or mesh networking, and may report summarized observations upward.
- 3) **Geographic scale:** kilometers to tens or hundreds of kilometers. Clusters may be separated by wide-area network links, cellular backhaul, fiber, radio, or intermittent connectivity. At this scale, the system should not rely on network latency to distribute precision time.

The core problem is not simply recording audio. Raw audio is voluminous, difficult to time-align after the fact, and expensive to move across battery-powered mesh networks. The core problem is building a time-disciplined acoustic observation system in which each layer converts its local measurements into a payload that preserves:

- the absolute time of the capture window,
- the geometry and uncertainty of the local sensor array,
- the likely bearing, elevation, frequency structure, and confidence of detected sources,
- enough compressed audio evidence to support later classification and audit,
- the metadata required to combine many local observations into a global spatial map.

V. TARGET TIMING BUDGET

The initial timing target is 1/64000 seconds:

$$T_{\text{target}} = \frac{1}{64000} = 15.625 \mu\text{s}.$$

At a 48 kHz audio sample rate, one sample period is:

$$T_s = \frac{1}{48000} \approx 20.833 \mu\text{s}.$$

At approximately 343 m/s, the speed of sound in air near 20°C, one audio sample corresponds to roughly:

$$d_s = 343 \text{ m/s} \times 20.833 \mu\text{s} \approx 7.15 \text{ mm}.$$

The 15.625 μs target is therefore smaller than one 48 kHz sample period, but not wildly smaller. It is a demanding requirement for ordinary software and ordinary networks, but it is plausible if time is treated as a hardware design constraint. This is the first architectural conclusion:

Precision acoustic mapping cannot depend on timestamping packets in user space after they arrive. Time must be captured at the edge of the physical measurement.

VI. LAYERED TIME CASCADE

The architecture uses a time cascade. Absolute time is established at large scale, distributed locally only where the physics and hardware support it, then used to trigger or clock local audio capture.

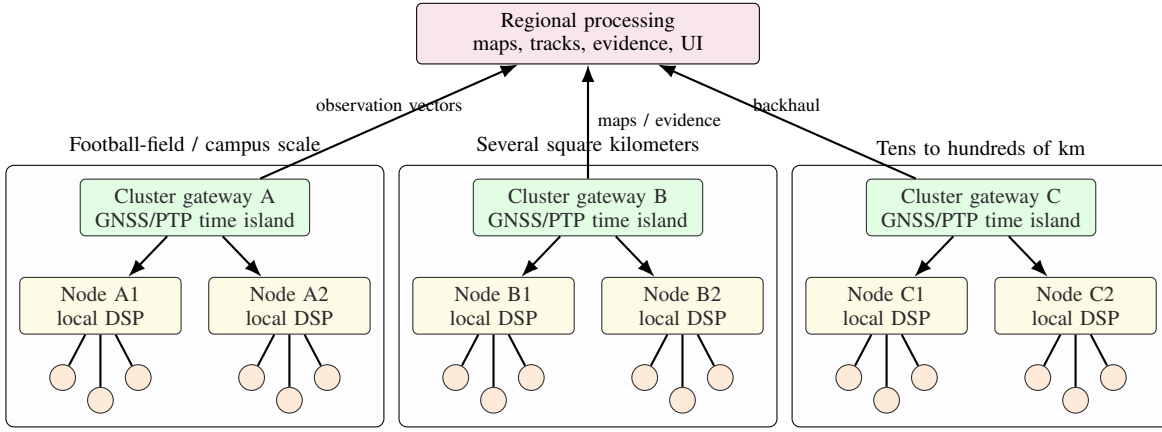


Fig. 1. Multi-scale deployment view. Local wired microphone groups preserve phase inside a node; node clusters communicate through a local gateway that owns the cluster time island; geographically separated clusters send compact observations and evidence references to regional processing.

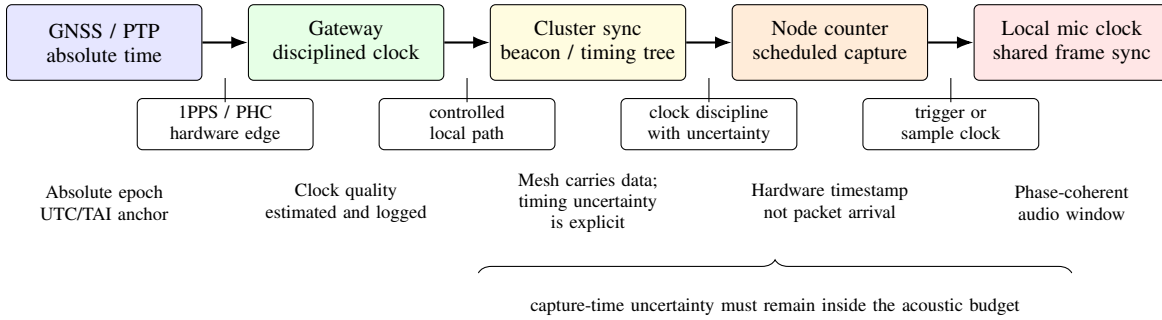


Fig. 2. Time-cascade view. Precision is established at each gateway, carried only over controlled local paths, and consumed by the node as a hardware capture clock or trigger. Application-level packet timing is not part of the measurement timestamp.

A. Geographic Tier

At the geographic tier, clusters are separated by kilometers or more. The obvious trap is to push time across the wide-area network. That should be avoided. Standard NTP is valuable for administrative time and log alignment, and NTPv4 is the established Internet clock synchronization protocol [4]. However, ordinary NTP across WAN, cellular, or public Internet paths should not be treated as a source of deterministic microsecond phase alignment.

Each cluster gateway should instead be treated as an autonomous time island. The gateway obtains absolute time locally, preferably from a GNSS receiver with a hardware time-pulse output. A GNSS module such as the u-blox ZED-F9P class can provide a configurable time pulse with tens-of-nanoseconds class specification in its data sheet and integration guidance [5], [6]. The exact module is not yet a final selection; the architectural requirement is a stable absolute time source with a hardware pulse or equivalent hardware timestamping path.

Where Ethernet distribution is needed inside a gateway cabinet or wired local segment, the system should prefer Precision Time Protocol with hardware timestamping. IEEE 1588 defines PTP for precision clock synchronization [7]; LinuxPTP is the common Linux implementation path for PTP and PHC-

aware systems [8]. This is a gateway/local-infrastructure tool, not a magic cure for arbitrary routed networks.

B. Cluster Tier

At the cluster tier, nodes may be separated by hundreds of meters and may communicate over Wi-Fi or a mesh topology. The design should exploit low-level Wi-Fi timing where available instead of trying to synchronize by application-level packets. IEEE 802.11 systems include a Timing Synchronization Function, and ESP-IDF exposes Wi-Fi and I2S APIs relevant to this class of implementation [9], [10].

The safe conceptual rule is:

Use mesh networking for communication, but do not let arbitrary multi-hop packet timing define measurement time.

A practical cluster design should enforce a strict time tree. A gateway is the root. Nodes accept time beacons or timing discipline from a known parent closer to the gateway. If a node is too many hops away to keep timing uncertainty inside the capture budget, the deployment should add another gateway or repeater rather than pretending that more mesh hops are free.

C. Local Sensor Tier

At the local tier, sensors are separated by meters and may be wired to a node. This is where the hard timing problem

becomes easiest. The local node can distribute a shared clock, frame sync, or trigger over copper. The crucial rule is to separate capture from harvest.

The wrong pattern is:

```
read_sensor_1();
timestamp_sensor_1();
read_sensor_2();
timestamp_sensor_2();
```

This stamps bus access, not acoustic reality. The right pattern is:

- 1) The node's hardware-timed counter reaches a scheduled capture instant.
- 2) A shared trigger or audio frame clock causes all microphones or ADC channels to sample together.
- 3) Samples are stored in local buffers.
- 4) The processor later reads the buffers and attaches the common capture timestamp.

For digital microphones or audio ADCs, this suggests an externally disciplined audio clock, such as a 12.288 MHz master clock divided down to 48 kHz frame sync, or a digital microphone bus in which the node controls the relevant clock. The engineering intent is not that every first prototype must be perfect; it is that local audio phase must be anchored in hardware, not in a scheduler.

VII. AUDIO CAPTURE CONCEPT

The baseline audio capture concept uses 48 kHz sampling, likely with 24-bit internal samples even if the transmitted feature payload is much smaller. The local node captures one-second or sub-second windows from a wired microphone group, then computes local acoustic features.

For drone detection, the useful acoustic content is expected to include blade-pass frequency, harmonics, broadband propeller noise, and motor/electronic signatures. Public UAV acoustic datasets and studies provide useful grounding material for later enrichment and validation, including UAV localization and classification datasets [11], [12], [13]. The concept does not assume that a single frequency bin identifies a drone. It assumes that the system can produce repeatable time-aligned observations, then combine:

- direction-of-arrival estimates,
- frequency and harmonic structure,
- signal-to-noise measures,
- time evolution and Doppler-like changes,
- compressed snippets for later classification.

A. Local Microphone Geometry

One plausible starting point is a five-microphone three-dimensional array: four microphones on a square base, plus one elevated center microphone. The purpose is to avoid a purely planar array, because planar arrays can create front/back or elevation ambiguities. A small tetrahedral or raised-cross geometry is a plausible starting point.

The exact geometry should be treated as a prototype parameter. It should be captured in calibration metadata:

- microphone positions in the local node frame,

- microphone orientation and acoustic port direction,
- measured channel delays,
- temperature and humidity context,
- enclosure and windscreen characteristics.

B. Local Signal Processing

The node should not normally forward all raw multichannel audio. Instead, it should act as a spatial encoder. For each capture window, local processing can include:

- 1) DMA capture of synchronized channels.
- 2) Windowing and FFT.
- 3) Band selection for drone-relevant, traffic-relevant, aircraft-relevant, or general acoustic events.
- 4) Time-difference-of-arrival estimation using generalized cross-correlation. The classic generalized correlation method for time-delay estimation is Knapp and Carter's 1976 paper [14].
- 5) Local beamforming or direction-of-arrival estimation.
- 6) Confidence estimation, including angular variance.
- 7) Optional local compression of a beamformed mono evidence snippet.

This keeps the network payload small while preserving spatial information.

VIII. ACOUSTIC OBSERVATION VECTOR

The central data product from a node should be an Acoustic Observation Vector. This is not yet a final schema, but it expresses the right boundary between local sensing and cluster/global fusion.

Explainer: acoustic signature vector versus observation vector

An **acoustic signature vector** is the compact, machine-readable description of what the local audio window sounded like after local signal processing. It is a feature vector: spectral energy by band, dominant frequencies, harmonic structure, modulation cues, SNR, DOA/TDOA-derived angles, classifier scores, and uncertainty measures. It is called a “vector” because it turns a large waveform into a bounded set of typed values that can be compared, fused, tracked, and stored.

Its lineage is general sensor design, not a new acoustic invention. Sensors commonly convert raw measurements into calibrated measurements, features, attributes, detections, and tracks. Multisensor fusion literature describes this broader pattern as combining data and related information from multiple sensors using techniques from pattern recognition, statistical estimation, and other fields [15]. Tracking-system literature similarly distinguishes measurements, attributes, data association, state estimation, and track fusion [16]. In this project, the acoustic signature vector is the acoustic-domain analogue of those feature and attribute reports.

The **Acoustic Observation Vector** proposed here is larger than the signature vector. It wraps the acoustic signature with capture time, node identity, node pose, array geometry, clock quality, environmental context, uncertainty, and evidence pointers. That wrapper is what lets downstream software ask, “what was heard, where was the sensor, when was it captured, how trustworthy was the clock, and how should this observation contribute to a map?”

This is therefore a **domain-specific adaptation of a general sensor-fusion idea**, not an implementation of a well-defined industry standard. It should be versioned and treated as an internal interface contract until later work either maps it onto an external standard, publishes it as a project schema, or replaces it with a better validated message model.

Listing 1. Conceptual Acoustic Observation Vector

```
{
  "schema": "acoustic_observation_vector.v0",
  "node_id": "cluster_B_node_04",
  "cluster_id": "cluster_B",
  "capture_start_tai_ns": 1782043289000000000,
  "capture_duration_ns": 1000000000,
  "node_origin_ecef_m": [324510.12, 541120.88, 1204.15],
  "node_pose_quat": [1.0, 0.0, 0.0, 0.0],
  "clock": {
    "source": "cluster_gateway_gnss_pps",
    "estimated_offset_ns": 3650,
    "estimated_drift_ppm": 1.0
  },
  "environment": {
    "temperature_c": 12.4,
    "humidity_percent": 74.0,
    "wind_speed_mps": 3.1,
    "wind_direction_deg": 210.0
  }
}
```

```
},
"detections": [
  {
    "source_track_hint": 1,
    "azimuth_deg": 42.5,
    "elevation_deg": 12.1,
    "angular_variance_deg2": 3.2,
    "dominant_freq_hz": 420.0,
    "band_energy": {
      "100_300_hz": 0.44,
      "300_1000_hz": 0.78,
      "1000_8000_hz": 0.61
    },
    "snr_db": 18.4,
    "evidence_audio": {
      "codec": "opus",
      "bitrate_bps": 32000,
      "payload_ref": "blob://..."
    }
  }
]
}
```

This payload does not say, “I recorded sound.” It says, “At this absolute time and place, with this estimated timing quality, I observed a probable source along this bearing and elevation, with this uncertainty and acoustic signature.”

IX. SPATIAL FUSION CONCEPT

At gateway or regional level, the system receives observation vectors from many nodes. The desired output is an acoustic map: where sound sources likely are, how confident the system is, what spatial resolution applies, and how the estimates evolve over time.

A. Log-Likelihood Voxel Grid

The proposed representation is a spatial log-likelihood voxel grid. A region is divided into voxels. Each observation contributes a probabilistic cone or ray bundle through that grid. Voxel values accumulate in log space:

$$L(x, y, z) \leftarrow L(x, y, z) + \log P_i(x, y, z),$$

where $P_i(x, y, z)$ is the probability contribution implied by node i ’s bearing, elevation, variance, and acoustic evidence.

This representation matches the desired “additive” structure. Independent observations do not need to be merged into a single fragile line crossing. Each observation warms up a spatial region; intersections accumulate higher likelihood.

B. Resolution Map

The map should not report only point estimates. It should report spatial resolution and uncertainty. A tight intersection between two or more well-separated viewing angles produces a compact likelihood peak. A shallow intersection produces a long smeared peak. The system should therefore compute:

- peak location,
- peak confidence,
- local covariance or Hessian-derived shape,
- cross-range and down-range uncertainty,
- number and geometry of contributing nodes,
- environmental corrections applied.

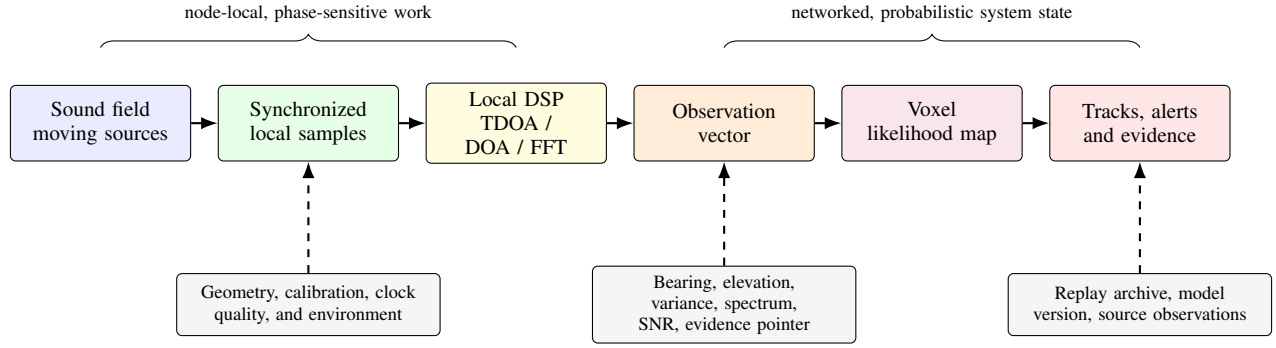


Fig. 3. Information-flow view. The node converts synchronized audio into compact spatial observations; the gateway or regional processor combines those observations into a likelihood map and temporal tracks while preserving metadata for replay and audit.

The output should be closer to “source at this coordinate with this uncertainty ellipsoid” than “source exactly here.”

C. Movement and Particle-Filter Tracking

The voxel map describes where sources are likely to be during one time window. Moving targets require a temporal layer on top of that map. The corrected concept is not a graphics-style particle system, but a particle filter: a sequential Monte Carlo tracker that represents a target’s possible states as weighted particles and updates those particles as new observations arrive.

This is a good conceptual fit for drones, vehicles, and aircraft because acoustic observations may be intermittent, nonlinear, multimodal, and non-Gaussian. A target state can include position, velocity, heading, altitude, acoustic class, and confidence. Each new voxel peak or observation vector reweights the particles according to how well the predicted target state explains the new evidence. Particle filters are well established for nonlinear and non-Gaussian Bayesian tracking problems [17], [18].

For this concept, the system can be read as three stacked probabilistic layers:

- 1) **Observation layer:** local microphone arrays estimate bearing, elevation, spectral features, and uncertainty.
- 2) **Spatial layer:** the gateway fuses observations into a voxel log-likelihood map.
- 3) **Temporal layer:** a tracker, likely a particle filter for early prototypes, estimates target motion and identity over time.

D. Environmental Corrections

The concept must account for the medium. Sound propagation changes with temperature, humidity, wind, terrain, and reflections. At this phase, the important design point is simple: environmental state belongs in the observation and fusion models. A node or gateway should not hard-code a single speed of sound. Temperature and humidity sensors are inexpensive enough to include on nodes, and wind sensors may be needed at gateways or representative points in larger deployments.

X. GPU PROCESSING CONCEPT

The gateway and regional fusion computations are naturally parallel. A grid of tens of millions of voxels, updated from dozens of node observations, is a poor match for a single sequential CPU loop. It is a good match for GPU kernels. CUDA is NVIDIA’s parallel computing platform and programming model for running compute-intensive code on GPUs [19]. Other back ends such as Vulkan Compute, OpenCL, SYCL, or CPU SIMD can remain future portability options.

A. Host Responsibilities

The host process should handle orchestration:

- receive observation vectors over UDP, QUIC, MQTT, or another selected transport,
- validate schema, clock metadata, node identity, and pose,
- normalize timescales, preferably avoiding UTC leap-second ambiguity in internal math,
- place observations into time-windowed ring buffers,
- push compact observation arrays to GPU memory,
- launch voxel update and peak extraction kernels,
- publish tracks, maps, alerts, and evidence references.

A pragmatic implementation split is a Go or Rust ingest service plus CUDA C++ kernels. Python can remain useful for offline analysis, training, and visualization, but it should not be assumed to be the real-time ingestion path.

B. Device Responsibilities

On the GPU, one natural kernel assigns one thread to one voxel or one small voxel block. Each thread computes its world coordinate, evaluates the active observation cones, accumulates log-likelihood, and writes the result.

Listing 2. Conceptual GPU kernel structure

```

__global__ void evaluate_acoustic_grid(
    float* voxel_grid,
    const NodeDetection* detections,
    int num_detections)
{
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    int z = blockIdx.z * blockDim.z + threadIdx.z;

```


Explainer: why log-likelihoods add

The short version is: independent evidence multiplies, but log evidence adds. Let ℓ be a candidate source location and let o_1, o_2 be observations from two independent nodes. The ordinary probability-space update has the form

$$P(\ell \mid o_1, o_2) \propto P(\ell)P(o_1 \mid \ell)P(o_2 \mid \ell).$$

That multiplication is mathematically correct, but it is awkward for a voxel grid. Multiplying many small probabilities can underflow numerically, and each new observation would require repeatedly multiplying every voxel by another small number. Taking logs turns the same update into:

$$\log P(\ell \mid o_1, o_2) = C + \log P(\ell) + \log P(o_1 \mid \ell) + \log P(o_2 \mid \ell),$$

where C is a normalization constant that can often be ignored while comparing voxels. This is why log-likelihood space is pleasant for this system: every node can contribute an additive “vote” to every compatible voxel. A weak observation adds a small amount over a wide region; a precise observation adds a sharper contribution; several observations agreeing in the same place create a strong peak by addition.

The caveat is important: the addition is only as valid as the likelihood model. If two observations share the same systematic error, for example wind bias, bad node geometry, or a common clock error, they are not truly independent. The system must therefore carry uncertainty and provenance rather than blindly treating every observation as a fresh independent vote. In practice, the fusion layer can down-weight correlated observations, include environmental corrections, or inflate uncertainty before adding the log-likelihood contribution.

```
if (x >= GRID_W || y >= GRID_H || z >=
    GRID_D) return;

float3 p = voxel_to_world(x, y, z);
float score = 0.0f;

for (int i = 0; i < num_detections; ++i) {
    score += log_likelihood_contribution(p,
        detections[i]);
}

voxel_grid[index3(x, y, z)] = score;
}
```

Later kernels can perform thresholding, 3D max-pooling, connected-component clustering, temporal association, and track filtering.

XI. PHYSICAL SYSTEM ELEMENTS

A. Local Acoustic Node

A first concrete node concept is a battery-powered acoustic puck with a small three-dimensional digital microphone array:

- **Microphones:** digital MEMS microphones or a multi-channel audio ADC path. Candidate part families include TDK/Invensense and Infineon digital MEMS microphones. Exact part selection requires later verification of sensitivity, self-noise, acoustic overload, supply current, bus topology, and outdoor packaging.
- **Geometry:** five microphones in a raised three-dimensional array, not a flat-only board.
- **Processor:** an ESP32-S3-class microcontroller is a plausible prototype platform because ESP-IDF supports Wi-Fi and I2S peripherals [9], [10]. It may not be the final platform if timing, memory, channel count, power, or DSP load exceed practical limits.
- **Clocking:** local capture clock derived from the best available cluster time discipline, with a stable sleep clock for duty-cycled operation.
- **Power:** LiFePO4 battery and solar charging are preferred for outdoor safety and cycle life.
- **Mechanical:** weatherproof enclosure, acoustic wind-screen, thermal path, insect/dust protection, and mechanical isolation between switching power electronics and microphones.

B. Cluster Gateway

A cluster gateway is both a communication concentrator and a time root. It should include:

- GNSS receiver with hardware time pulse,
- stable oscillator or disciplined clock path,
- Linux compute module or industrial PC,
- Wi-Fi, Ethernet, or radio interfaces for local nodes,
- optional PTP-aware Ethernet interface or switch,
- local storage for evidence buffers,
- backhaul to a regional server,
- environmental sensors, especially wind and temperature where useful.

For small deployments, the gateway might also run the spatial fusion workload. For larger deployments, it may forward observation vectors to a GPU-equipped regional node.

C. Regional Processing Node

The regional processing node runs the voxel map, classifier, track manager, and operator interface. A field-deployed system might use an NVIDIA Jetson AGX Orin-class device; a fixed site might use an industrial PC with a workstation GPU. This is not yet a procurement recommendation. It is a sizing hypothesis: the workload is GPU-shaped once the grid and node count grow.

XII. PCB AND DESIGN FILE STRATEGY

The design-file strategy should support production-intent mixed RF, digital audio, low-power, and outdoor sensor hardware. Informal breadboard diagrams are not enough for a

microphone-array node that combines high-speed digital audio lines, a Wi-Fi radio, switching power, solar charging, environmental sensors, and weatherproof mechanical constraints.

The project should move toward:

- **KiCad** for open-source schematic and PCB design, including schematic capture, PCB layout, 3D/mechanical checks, Gerber viewing, SPICE support, and repeatable fabrication exports [20],
- **EasyEDA** where tight integration with low-cost PCB assembly is useful for fast prototypes and the required components are available in the assembly catalog [21], [22],
- explicit design rules for ground planes, return paths, digital microphone routing, antenna keep-outs, switching-regulator isolation, and connector sealing,
- a versioned library strategy for symbols, footprints, and mechanical models,
- manufacturing outputs archived with each prototype revision,
- calibration fixtures and firmware hooks included in the design from the beginning.

Open-source ESP32 and MEMS microphone projects may be useful as starting points, but individual repositories should be verified later before they are used as design inputs. OSHWLab is useful as a searchable source of EasyEDA hardware examples because it exposes categories for audio, ESP8266/ESP32, Wi-Fi/Ethernet, power, RF, high-speed, mixed-signal, EMC/EMI, and sensor-module designs [23]. This document does not treat any unverified community project as an authoritative reference.

A. Prototype Acceleration Workflow

The hardware design tools should be used to reduce elapsed prototype time without losing engineering traceability. A practical first-delivery path is:

- 1) **Separate the prototype into boards:** create one microphone-array board, one compute/power carrier, and one gateway timing interface before combining them. This keeps acoustic geometry, noisy power conversion, radio placement, and timing bring-up independently testable.
- 2) **Start with catalog parts and proven footprints:** constrain the first spin to parts with available footprints, assembly availability, and accessible reference layouts. The goal of the first board is not elegance; it is to collect synchronized audio and expose timing errors early.
- 3) **Use OSHWLab and GitHub as design-pattern libraries:** search for ESP32, INMP441/ICS-43434-class microphones, battery/solar charging, low-noise regulators, and weatherproof connectors. Reuse footprints, connector conventions, and bring-up ideas only after checking licenses, revision history, and whether the design actually builds.
- 4) **Run a dual-track EDA flow:** keep the canonical schematic, layout, libraries, and mechanical models in KiCad; optionally mirror a stripped-down first-spin board

into EasyEDA/JLCPCB when assembly integration can shorten delivery.

- 5) **Design for bring-up from the first schematic:** add test pads for I2S clocks, frame sync, microphone data, PPS/time-pulse, battery rail, regulator outputs, reset/boot, and UART/JTAG. Include solder jumpers or zero-ohm links that allow the microphone clock tree, radio, charger, and processor to be isolated during debugging.
- 6) **Archive every fabrication artifact:** each prototype revision should preserve schematic PDF, native EDA files, Gerbers, drill files, BOM, pick-and-place, assembly notes, firmware commit, and measured bring-up results.
- 7) **Use non-drone acoustic targets for first validation:** cars, aircraft, mechanical noise sources, loudspeakers, and controlled moving emitters are easier first targets than drones. They let the team validate timestamping, geometry, array health, observation-vector generation, and replay tooling before drone-specific trials.

This creates a fast but disciplined path to initial delivery: a bench-verifiable microphone board, a fieldable node carrier, a gateway timing harness, and a replayable data pipeline can advance in parallel. The main guardrail is that the quick-turn flow must not hide the timing and noise risks. Every board should be judged by measured clock quality, self-noise, microphone health, power behavior, and whether it produces observation vectors that the software stack can replay.

XIII. INFORMATION AND CONTROL ARCHITECTURE

A. Node Firmware

Node firmware should be organized around deterministic capture and compact reporting:

- boot, identity, secure provisioning, and cluster association,
- clock discipline and timing-quality estimation,
- scheduled capture windows,
- I2S or ADC DMA buffer management,
- local FFT and GCC-PHAT/TDOA processing,
- detection candidate extraction,
- evidence snippet compression,
- observation vector emission,
- duty-cycle and battery management,
- local diagnostics for clock lock, microphone health, noise floor, temperature, and supply voltage.

B. Gateway Software

Gateway software should bridge field hardware and spatial inference:

- GNSS and clock-status monitoring,
- node registry and pose management,
- observation ingestion,
- local storage and replay,
- health telemetry,
- network backhaul,
- optional local voxel fusion for small clusters,
- forwarding to regional GPU processing.

C. Fusion and UI Software

Fusion software should maintain time-windowed acoustic maps and tracks:

- per-window voxel likelihood grids,
- temporal smoothing and track association,
- classifier hooks for drones, aircraft, vehicles, wildlife, machinery, and unknown events,
- evidence retrieval from nearest nodes,
- map visualization with uncertainty ellipsoids,
- alerting policies,
- audit logs that preserve source observations and model versions.

XIV. TESTING WITHOUT DRONES

Drone-free testing is not only possible, it is desirable. Drones are difficult early test targets because they are small, short-duration, battery-limited, and operationally constrained. The concept can be tested with easier sources first.

A. Bench and Yard Tests

A speaker on a tripod can emit sine sweeps, chirps, broad-band noise, or drone-like synthetic tones. Moving the speaker through a known path verifies:

- local direction-of-arrival estimation,
- timestamp alignment between nodes,
- voxel peak motion,
- evidence snippet capture,
- map UI behavior.

B. Traffic Tests

Road traffic provides moving acoustic sources constrained to known paths. Cars and trucks should generate tracks that follow road geometry. If the acoustic map consistently drifts away from the road, the system has a geometry, timing, wind, or propagation-model error.

C. Aircraft Tests

Aircraft provide loud, trackable, externally observable sources. ADS-B receivers can provide independent ground truth for many aircraft, though acoustic propagation delay and atmospheric effects must be handled carefully. Aircraft tests are useful for validating large-scale geometry, not for pretending aircraft sound behaves exactly like low-altitude drone sound.

XV. KNOWN ENGINEERING TRAPS

This phase is not a critique phase, but several obvious traps should be preserved because they shape the concept:

- **Router GNSS is not automatically precision time:** a router that parses GNSS NMEA and serves NTP may be fine for logs, but it is not automatically a microsecond-grade phase reference.
- **User-space timestamps are too late:** timestamping after packet receipt or bus reads measures software behavior, not acoustic capture time.
- **Mesh timing is not free:** data mesh and time distribution should be treated separately.

- **Raw audio does not scale:** local nodes should summarize and compress.
- **Planar arrays are ambiguous:** three-dimensional microphone geometry is strongly preferred.
- **Outdoor acoustics are hostile:** wind, rain, insects, reflections, ground bounce, temperature gradients, and self-noise will matter.
- **UTC can be awkward internally:** for internal continuous timing, TAI-like monotonic timescales avoid leap-second ambiguity. UTC can be presented at the UI boundary.

XVI. CONCEPT-TO-PRD SEEDS

The eventual PRD can be built around these capability groups:

- 1) **Timing:** define required absolute and relative timing error by tier.
- 2) **Capture:** define sample rate, channel count, bit depth, and phase-lock requirements.
- 3) **Node hardware:** define power budget, enclosure, microphone geometry, processor, storage, and radio.
- 4) **Gateway hardware:** define GNSS, time pulse, backhaul, local compute, and environmental sensing.
- 5) **Observation schema:** define the Acoustic Observation Vector format.
- 6) **Fusion:** define voxel resolution, update rate, uncertainty reporting, and GPU sizing.
- 7) **Classification:** define drone and non-drone classes, confidence outputs, and evidence retention.
- 8) **Deployment:** define field calibration, node placement, health monitoring, and maintenance.
- 9) **Validation:** define bench, traffic, aircraft, and drone test procedures.
- 10) **Security:** define node identity, signed firmware, encrypted transport, and tamper handling.

XVII. CONCLUSION

The central concept is a layered acoustic sensing architecture in which time quality is preserved at every boundary. Geographic clusters should obtain absolute time locally. Cluster nodes should use hardware-assisted timing where possible and avoid treating arbitrary mesh packet paths as timing references. Local microphones should be clocked or triggered in hardware. Nodes should convert raw audio into compact acoustic observation vectors. Gateways and regional processors should combine those vectors into an additive spatial likelihood map, report uncertainty as a first-class output, and use GPU acceleration where grid size and node count justify it.

This gives the future PRD a clear skeleton: time-disciplined local capture, compact spatial observations, additive map fusion, and prototype hardware that can be tested incrementally before drone-specific trials.

ABOUT THIS DOCUMENT

This document represents a technical exploration created through a collaborative process between human and AI. The production process followed these steps:

- 1) **Discovery:** The topic emerged from a systems design exploration of distributed acoustic sensing across local, cluster, and geographic scales.
- 2) **Initial Exploration:** The initial exploration covered clock synchronization, Wi-Fi and mesh timing, wired microphone groups, acoustic mapping, GPU processing, drone-oriented hardware, PCB tooling, and drone-free validation targets.
- 3) **Synthesis:** The material was organized into a concept document with explicit problem definition, engineering constraints, layered architecture, data products, hardware concepts, software concepts, and PRD seed areas.
- 4) **Implementation:** No implementation was produced in this phase. The document is intended to guide later requirements, prototypes, and verification work.
- 5) **Visualization:** Diagrams were added for the executive summary, multi-scale deployment topology, timing cascade, and observation-to-fusion information flow. Future versions should add more detailed node hardware, PCB bring-up, and voxel-fusion diagrams as implementation detail increases.
- 6) **Verification:** References included in the bibliography were checked for URL accessibility and for basic correspondence between the cited source and the cited claim.

This document is intended for system design planning. It should be enriched later with measured timing budgets, verified component data, propagation models, safety analysis, cost estimates, and prototype test results.

NOTE ON REFERENCES AND VERIFICATION

This document contains AI-generated content. All references have been subject to verification checks intended to support academic and engineering integrity.

Verification Process:

- URLs were tested for accessibility using web lookup tools.
- Author names and organizations were checked against the linked source metadata where available.
- DOIs were included where found for standards or papers.
- Publication venues and official documentation sources were preferred.
- Content relevance was checked against the claims supported by the citation.

Verification Status in References: Each reference includes a note field indicating its verification status:

- “Verified: URL accessible” – URL was tested and works.
- “Verified: DOI accessible” – DOI or DOI-backed landing page was confirmed.
- “Official documentation” – vendor, standards, or project documentation was used.

Important Notice: Due to the AI-assisted nature of this document’s creation, readers should independently verify technical claims before using them for procurement, safety-critical deployment, regulatory work, or field operations.

REFERENCES

- [1] “Systems and software engineering – system life cycle processes,” ISO/IEC/IEEE, 2023, verified: ISO URL accessible; standard page describes life-cycle processes for systems and systems of systems. [Online]. Available: <https://www.iso.org/standard/81702.html>
- [2] International Council on Systems Engineering, “Systems engineering handbook,” 2023, verified: INCOSE URL accessible; handbook page describes systems engineering practice and ISO/IEC/IEEE 15288 alignment. [Online]. Available: <https://www.incose.org/resources-publications/technical-publications/se-handbook/>
- [3] NASA, *NASA Systems Engineering Handbook*, National Aeronautics and Space Administration, 2016, verified: official NASA PDF accessible; handbook covers systems engineering fundamentals, design processes, realization, and verification. [Online]. Available: https://www.nasa.gov/wp-content/uploads/2018/09/nasa_systems_engineering_handbook_0.pdf
- [4] D. L. Mills, J. Martin, J. Burbank, and W. Kasch, “Network time protocol version 4: Protocol and algorithms specification,” RFC Editor, Tech. Rep. RFC 5905, June 2010, verified: RFC Editor URL accessible; DOI and metadata identify the NTPv4 specification. [Online]. Available: <https://www.rfc-editor.org/info/rfc5905/>
- [5] u-blox, *ZED-F9P-04B Data Sheet*, u-blox AG, 2021, verified: official u-blox PDF accessible; data sheet for ZED-F9P high precision GNSS module and time pulse specifications. [Online]. Available: https://content.u-blox.com/sites/default/files/ZED-F9P-04B_DataSheet_UBX-21044850.pdf
- [6] —, *ZED-F9P Integration Manual*, u-blox AG, 2024, verified: official u-blox PDF accessible; integration manual includes timing and time-pulse design considerations. [Online]. Available: https://content.u-blox.com/sites/default/files/ZED-F9P_IntegrationManual_UBX-18010802.pdf
- [7] “Ieee standard for a precision clock synchronization protocol for networked measurement and control systems,” IEEE, 2020, verified: IEEE Xplore URL accessible; DOI matches IEEE 1588-2019 PTP standard. [Online]. Available: <https://ieeexplore.ieee.org/document/9120376>
- [8] Time-Sensitive Networking Documentation Project, “Synchronizing time with linux ptp,” 2026, verified: URL accessible; documents LinuxPTP tools including ptp4l, phc2sys, and PHC synchronization. [Online]. Available: <https://tsn.readthedocs.io/timesync.html>
- [9] Espressif Systems, “Wi-fi api reference, esp-idf programming guide,” 2026, verified: official Espressif documentation URL accessible; includes TSF time API behavior. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_wifi.html
- [10] —, “Inter-ic sound (i2s), esp-idf programming guide,” 2026, verified: official Espressif documentation URL accessible; describes I2S, PDM, sample clocks, and DMA buffer behavior. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/i2s.html>
- [11] G. Jekateryczuk, R. Szadkowski, and Z. Piotrowski, “Uavirbase: A public-access unmanned aerial vehicle sound source localization dataset,” *Applied Sciences*, vol. 15, no. 10, p. 5378, 2025, verified: MDPI URL accessible; article presents a public UAV sound source localization dataset. [Online]. Available: <https://www.mdpi.com/2076-3417/15/10/5378>
- [12] S. Al-Emadi, A. Al-Ali, and A. Al-Ali, “Audio-based drone detection and identification using deep learning techniques with dataset enhancement through generative adversarial networks,” *Sensors*, vol. 21, no. 15, p. 4953, August 2021, verified: Hamad Bin Khalifa University publication page accessible; metadata identifies Sensors article, DOI, authors, and drone acoustic dataset/enhancement topic. [Online]. Available: <https://elmi.hbku.edu.qa/en/publications/audio-based-drone-detection-and-identification-using-deep-learning-2/>
- [13] M. Mięsikowska, “Dataset of acoustic signals of unmanned aerial vehicles and speech obtained in external environmental conditions,” September 22 2023, verified: Zenodo URL accessible; dataset page describes UAV and speech acoustic signals recorded in external environmental conditions. [Online]. Available: <https://zenodo.org/records/15190811>
- [14] C. H. Knapp and G. C. Carter, “The generalized correlation method for estimation of time delay,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 4, pp. 320–327, August 1976, verified: NASA ADS URL accessible; DOI and metadata match classic generalized correlation time-delay estimation paper. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/1976ITASS..24..320K/abstract>
- [15] D. L. Hall and J. Llinas, “An introduction to multisensor data fusion,” *Proceedings of the IEEE*, vol. 85, no. 1, pp. 6–23, January 1997, verified: OSTI URL accessible; page lists title, authors, Proceedings of the IEEE, volume 85, issue 1, DOI, and abstract describing multisensor data fusion applications, process models, pattern recognition, and statistical estimation. [Online]. Available: <https://www.osti.gov/biblio/442687>
- [16] S. S. Blackman and R. Popoli, *Design and Analysis of Modern Tracking Systems*. Artech House, 1999, verified: official Artech House URL accessible; page lists authors, copyright year, ISBN, and contents covering sensor/source characteristics, tracking, attribute fusion, multiple-sensor tracking, situation assessment, and sensor management. [Online]. Available: <https://us.artechhouse.com/Design-and-Analysis-of-Modern-Tracking-Systems-P170.aspx>
- [17] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, “Novel approach to nonlinear/non-gaussian bayesian state estimation,” *IEE Proceedings F - Radar and Signal Processing*, vol. 140, no. 2, pp. 107–113, April 1993, verified: IET Digital Library URL accessible; DOI and metadata match foundational bootstrap particle filter paper. [Online]. Available: <https://digital-library.theiet.org/doi/10.1049/ip-f-2.1993.0015>
- [18] M. S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp, “A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking,” *IEEE Transactions on Signal Processing*, vol. 50, no. 2, pp. 174–188, February 2002, verified: IEEE Xplore URL accessible; DOI and metadata match particle filter tutorial for nonlinear/non-Gaussian Bayesian tracking. [Online]. Available: <https://ieeexplore.ieee.org/document/978374>
- [19] NVIDIA Corporation, “Cuda c++ programming guide,” 2026, verified: official NVIDIA documentation URL accessible; describes CUDA programming model for GPU computation. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>
- [20] KiCad Documentation Contributors, “Introduction,” 2026, verified: official KiCad documentation URL accessible; page describes KiCad 10.0 as a free/open-source EDA suite with schematic capture, SPICE simulation, PCB layout, 3D rendering, fabrication exports,

- Gerber viewer, and BOM generation. [Online]. Available: <https://docs.kicad.org/10.0/en/introduction/introduction.html>
- [21] JLCPCB, "From pcb design to pcba – easyeda and jlcpcb," 2026, verified: official JLCPCB URL accessible; page describes EasyEDA and JLCPCB as an integrated PCB design, PCB prototype, SMT assembly, and small-batch production flow. [Online]. Available: <https://jlcpcb.com/EDA>
- [22] EasyEDA, "How to order pcb," 2026, verified: official EasyEDA documentation URL accessible; page documents BOM export, Gerber generation, PCB ordering, SMT assembly prerequisites, BOM upload, and pick-and-place upload. [Online]. Available: <https://docs.easyeda.com/en/PCB/Order-PCB/>
- [23] OSHWLab, "Explore projects," 2026, verified: official OSHWLab project-index URL accessible; page lists categories relevant to ESP32, audio, Wi-Fi/Ethernet, RF, power, high-speed, mixed-signal, EMC/EMI, and sensor design scouting. [Online]. Available: <https://oshwlab.com/explore>